TITLE OF THE PROJECT

Hallucination-Resistant RAG for Government Information

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

DIVAN PARGI
Sch. No - 24U020093
Sem-5

Under the supervision of

Dr. Ringki Das
Assistant Professor (CSE)
(Computer Science and Engineering)

October & 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled **"Hallucination-Resistant RAG for Government Information"** has been satisfactorily completed by **DIVAN PARGI (24U020093)** It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Engineering during the academic year 2026-27

Date:21-10-2026

Dr.Ringki Das,
Assistant Professor (CSE),
Computer Science and Engineering,
IIIT Bhopal (M.P.)

Head/Department Coordinator,-
Dr. Rahul Chaurasia
Department of CSE,
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the following major project synopsis entitled **"Hallucination-Resistant RAG for Government Information"** presented in the is the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**. It is an authentic documentation of our original work carried out under the able guidance of.**Dr.Ringki Das**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Divan Pargi Sign of student
2**24U020093**

ACKNOWLEDGEMENT

We , **Divan Pargi**, would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled **"Hallucination-Resistant RAG for Government Information**".

First and foremost, we would like to extend our heartfelt thanks to our esteemed **Director** ,**Dr. Ashutosh Kumar Singh**, for providing us with the necessary resources and facilities for conducting our project. His/her encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our **Project Guide**, **Dr. Ringki Das**, for his/her constant guidance, invaluable suggestions, and expert advice throughout the project. His/her knowledge and insights helped us enhance the quality of our work, and his/her support kept us motivated during difficult times.

Our sincere thanks to the **Department Coordinator**, **Dr. Rahul Chaurasia**
, for the valuable support and encouragement provided during completion of the project.

We would also like to express our gratitude to all the **faculties** of the **Computer Science and Engineering** for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our **friends** for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Divan Pargi
24U020093

AREA OF WORK

Our project mainly revolves around the concept of Artificial Intelligence, Natural Language Processing, and Retrieval Augmented Generation. We focus on creating a question-answering system that can retrieve relevant information from documents and generate reliable answers.

The project includes using the Unstructured library and Tesseract OCR for document processing, BAAI/bge-m3 for creating semantic embeddings, Qdrant Cloud for storing vectors and retrieving information, and llm model for generating answers.

Through this project, we intend to learn how document processing, semantic search, and Large Language Models can work together to enhance information retrieval, provide source references, and reduce the likelihood of generating unsupported answers.

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	
	Declaration	
	Abstract	1
1	Introduction	2
2	Literature review or Survey	4
3	Problem definition and objectives	12
4	Methodology & Work Description	14
5	Proposed algorithm	18
6	Proposed flowchart/ DFD/ Block Diagram	21
7	Tools & Technology Used	22
8	Result Analysis	24
9	Conclusion & Future Scope	29
10	References	31

LIST OF FIGURES

Fig	Description	Page no.
1	Overview of the System	14
2	Architecture of the proposed system	21
3	Retrieval-Augmented Generation (RAG) workflow	21
4	Output 1	26
5	Output 2	26

LIST OF TABLES

Table No	Description	Page no.
1	Comparison of RAG and Fine-Tuning	6
2	Comparison of Approaches	9
3	Summary of Research Papers	11
4	Tools And Technology Used	22
5	Results of the testing of the system	25
6	Comparison with a Normal LLM	25

ABSTRACT

Large Language Models (LLMs) give very good answers to the questions that are covered by theirbytheirby their training data. But when a user asks a question from a private document, which the model has never seen, the LLM often gives an irrelevant or incorrect answer. This problem is called hallucination.

To solve this problem, Retrieval-Augmented Generation (RAG) is used. RAG makes the LLM answer from private documents. It works in a few stages. First, the data is collected and stored in a database to create a knowledge base. Then the user asks a question related to a particular domain, and the answer is given using the knowledge base together with the LLM. If the knowledge base does not have the required information, the system replies that it is not able to answer that question. This is the main idea of this project.

In this project, the PDF files are read using the unstructured library with OCR. The text is then split into chunks, and each chunk is converted into a vector using the BAAI/bge-m3 embedding model. These vectors are stored in the Qdrant vector database. When the user asks a question, the system retrieves the top-k chunks related to the question from Qdrant and gives them to the LLM. The LLM then generates the answer from these chunks.

Keywords: RAG, hallucination, LLM, vector database, embeddings,

INTRODUCTION

1.1 Background :

Every year, the Government of India and the state governments announce numerous scholarship schemes for students. The details of the schemes are provided in PDF files like notifications, guidelines and circulars. The files are unstructured and filled with tables, rules, and legal language. A normal student or user may find it difficult to locate specific information, such as the income limit or application deadline, in a 40-page PDF document

Moreover, the same scheme can have many documents. A new circular might change a rule from an older circular. Students and parents usually do not have the time to read everything and instead rely on friends, agents, or websites. Sometimes, the information is incomplete or outdated.

Chatbots based on Large Language Models seem to be an easy solution. A student can type a question in a simple language and receive the answer in a few seconds. However, a typical chatbot answers based on the training data. It does not read the latest government notification. so, it can provide outdated or wrong details.

1.4 What is Hallucination?

Every time we use an LLM, we need a proper and correct answer from it. But when I ask a question related to a private document, the LLM does not work properly and gives a random answer that only sounds correct. This problem is called hallucination.

Hallucination is not a rare problem. It happens many times with many users, so it is not a small problem, it is a big one. When we ask a question from some other domain, the LLM may give a wrong answer, and this is not acceptable. For example, if a student asks an LLM about the deadline of a scheme and the LLM gives a wrong date, the student may miss the real date. This is why hallucination is a major problem.

1.3 What is RAG?

Retrieval-Augmented Generation (RAG) is a process in which a document is converted into a knowledge base. This is done in the following steps:

- document loading
- text extraction
- chunking
- embedding
- vector database

These steps create the knowledge base from our own private documents. Then, when a user asks a question, the system finds the top-k related chunks from the knowledge base and gives them to the LLM as context, along with some rules. The LLM then writes the answer only from these related chunks. In this way the hallucination problem is reduced.

If the answer is not present in the knowledge base, the system does not guess. It replies with a message like “I am not able to answer this question”. So in RAG, the system uses its own knowledge base, which is made from the private documents, and the model answers the ques tions from these documents

1.4 Motivation

The reason for selecting this topic is that large language models (LLMs) are used extensively in many fields; however, these are prone to providing false information and, for a broad domain of application, a high accuracy in responses is required. To solve this problem, I examined the inner workings of LLMs and investigated various approaches to addressing the issue of hallucination, as well as RAG methodology that is helpful in constructing a reliable response on the basis of sources. Therefore, I selected the topic for investigation because it has a practical application in everyday life by providing the higher degree of response relevance and reliability.

1.5 Scope of the Project

This project focuses on retrieving and understanding information from PDF documents using Retrieval-Augmented Generation (RAG). It helps users find specific information, such as eligibility criteria, income limits, required documents, and application deadlines, without manually searching through lengthy documents. The system retrieves relevant information from the stored documents and generates answers based on the retrieved content, along with source references. The main objective is to make information retrieval easier and reduce the time required to find relevant information in large PDF documents.

1.6 Organization of the Report

The rest of the report is organized as follows. Section 2 presents the literature survey. Section 3 explains the basic concepts of Retrieval-Augmented Generation (RAG). Section 4 discusses the challenges associated with processing and retrieving information from PDF documents. Section 5 describes the proposed methodology and implementation. Section 6 presents the results and discussion. Section 7 concludes the report, and Section 8 discusses the future scope of the project.

LITERATURE REVIEW

In this section, I describe the related research and concepts which I explored before starting this project. The survey includes hallucination in language models, Retrieval-Augmented Generation (RAG), retrieval techniques, document processing, and evaluation of generated answers

2.1 Hallucination in Language Models

Ji et al. [2] conducted the survey on the phenomenon of hallucination in natural language generation. They discussed the types of hallucination: intrinsic and extrinsic. Intrinsic hallucination occure when the generated output not correct from the given source, and extrinsic hallucination is the inclusion of information that cannot be verified from the source. They identified several root causes of hallucination, including training data limitations, incorrect patterns learned by the model, and language modeling issues.

The critical problem of hallucination in my project is the generation of unsupported information. For example, a student may ask a question about the rules of a scholarship which is not described in the given documents. A language model may generate an answer with the information that it collected from training data, but this answer will not be relevant to the particular question. Thus, the system should make it clear that the required information was not found in the given documents

2.2 Retrieval-Augmented Generation

Lewis et al. [1] developed Retrieval-Augmented Generation (RAG) for knowledge-intensive tasks. According to the article, RAG uses a retriever to find relevant chunks in the knowledge source and give to llm/ generator to create the final answer based on the retrieved information. The study demonstrated the effectiveness of RAG for such tasks.

An essential advantage of RAG is that the knowledge base can be updated by adding new documents or modifying existing ones. This feature can be extremely beneficial for government documents schemes and notifications, which are frequently updated

Gao et al. [3] conducted the survey of RAG methods, and identified three main approaches: Naive RAG, Advanced RAG, and Modular RAG. Naive RAG is the baseline, which employs indexing, retrieval, and generation. Advanced RAG introduces several modifications, includ ing query rewriting, improved chunking, and re-ranking. In Modular RAG, the system has a modular structure, and each module can be replaced or refined

The survey of Sharma [4] offered several insights for my project. First, the quality of the re trieved documents has a significant impact on the final result, which aligns with my motivation to carefully process the given PDF files. Second, the retrieval step is not sufficient for gener ating supported answers, which explains the use of SELF-RAG and Corrective RAG (CRAG) methods. Third, a small collection of relevant documents can ensure that the model will use information from this collection, and my project uses the government documents as the knowl edge source. Finally, the survey emphasizes the importance of filtering out irrelevant evidence and refusing to answer when the required information is not found, which I also consider in my project.
Based on the survey, I decided to implement the retriever-based RAG and add instructions to the language model to prevent hallucination

2.3 Self-Checking and Corrective Methods

Many articles offer solutions to improve RAG by introducing checks of the retrieved evidence or generated answers.

Asai et al. [10] developed the SELF-RAG method, which allows the model to decide whether to perform the retrieval step or not and to evaluate the generated answer with specially designed signals.

Yan et al. [11] proposed Corrective RAG (CRAG), which applies an evaluator to assess the quality of the retrieved documents. If the retrieved evidence is of low quality, the system can take corrective measures, such as conducting another retrieval step

SELF-RAG and CRAG were not used in my project, as the system is a simple demonstrator of the RAG concept. However, these methods address the issue of unsupported answers by checking the evidence before or after the answer is generated. My project implements a similar approach, as the model should not rely on its own knowledge and should refuse to answer if the required evidence is not found

2.4 Position of Information in the Context

Liu et al. [12] investigated the impact of position of information on the performance of language models. Through the experiments, they identified that the model can have different accuracy for the same input, depending on where the relevant Information is located. For some tasks, the position of relevant information affects the answer: the model may use the information at the beginning or the end of the input, while the middle part may be ignored. This effect is known as the “Lost in the Middle” phenomenon

This result is especially relevant to the RAG systems, as the retrieved passages are placed in the prompt, which the language model processes. Adding more retrieved chunks to the prompt may not improve the result, and the model may fail to use the information which was placed in the middle of the prompt. Therefore, the number of retrieved chunks should be sufficiently small, and the important information should be prioritized. In my project, the approach of using relevant chunks rather than repeated information aligns with this assumption

2.5 RAG versus Fine-Tuning

Fine-tuning is the process of additional training of the language model to make it suitable for a particular task or domain. It can be applied together with RAG to improve the results, but the two approaches have significant differences, as shown in Table 1.

In the context of the government scholarships, the information may change, and the RAG system can easily update the documents in the knowledge base. In contrast, fine-tuning would require additional training of the model each time the documents are updated. Another ad vantage of RAG is that it explicitly retrieves relevant information from the documents, while fine-tuning does not inherently support this capability. Based on these considerations, the RAG approach is more suitable for this project, as the system should provide answers based on the current government documents

Table 1: Comparison of RAG and Fine-Tuning

Comparison Point	Retrieval-Augmented Generation (RAG)	Fine-Tuning
Main purpose	Provides external information during answer generation	Adapts model behaviour or knowledge through additional training
Knowledge updates	Documents can be updated in the knowledge base	New training or adaptation may be required for new information
Training requirements	Does not require retraining the LLM for each document update	Requires additional training for fine-tuning
Source references	Can retrieve source passages and metadata for citations	Does not provide citations to source documents
Information retrieval	Can retrieve relevant information for the given query	Does not inherently retrieve external documents

Comparison Point	Retrieval-Augmented Generation (RAG)	Fine-Tuning
Cost of frequent updates	Can be relatively low when only the document collection changes	Can be higher when repeated fine-tuning is needed
Main limitation	Depends on retrieval quality and relevance of context	Knowledge updates and source-level verification may be difficult
Suitability for this project	Suitable for frequently updated government documents	Less suitable when the main requirement is to retrieve current information from official documents

2.6 Evaluation Tools and Datasets

An essential part of developing the RAG system is the evaluation of its performance. The re trieved information can be relevant, but the model may generate an incorrect answer. Similarly, the model may generate a correct answer, but it may not properly cite the source information.

Es et al. [13] developed the RAGAs framework, which provides several methods to evaluate RAG systems on different aspects of performance. Some of these methods do not require a reference answer, which is beneficial when correct answers are not available for the questions.

Niu et al. [14] introduced the RAGTruth dataset, which consists of RAG responses annotated with hallucination labels. It can be used to evaluate and improve the RAG systems' ability to detect unsupported information.

Chen et al. [15] developed the RGB benchmark, which tests the main aspects of RAG systems, such as robustness to noisy information, refusal to answer when evidence is not found, the ability to combine information from multiple sources, and the robustness to misinformation.

One of the critical requirements for this project is the ability to refuse to answer when the evidence is not found, as it will reduce the risk of providing false or misleading information. According to the results of the RGB benchmark, the answer validation is a challenging task, and the model may fail to detect that the required information is not present in the retrieved documents. The methods described in Section 2.3 can explicitly address this problem

2.7 Dense Retrieval and Embeddings

Traditional information retrieval systems typically use the Term Frequency– Inverse Docu ment Frequency (TF-IDF) and BM25 algorithms, which are based on the comparison of words. These algorithms work correctly when the words in the query match the words in the docu ments, but they do not account for possible word substitutions. For example, the query may contain the phrase ‘fee waiver’, while the document describes the same concept as ‘tuition fee exemption’.

Karpukhin et al. [5] developed a system for dense passage retrieval, in which the query and the documents are converted into a dense representation. The two vectors can be compared even if the documents do not explicitly contain the words from the query.

The BAAI/bge-m3 embedding model [6] can be used for multilingual dense retrieval. It is suitable for this project, as the government documents can be in English or Hindi.

In this project, the BAAI/bge-m3 model is used to encode the document chunks and the user’s question into the embedding vectors, which are compared to find the most relevant document chunks

2.8 Vector Databases

The vector database is optimized for similarity search of embedding vectors. It also stores metadata about the embeddings, which can be used to filter the results and find the required document.

Qdrant [16] is an open-source vector database, which can be used to store the document embed dings and perform similarity search. It provides the filtering of results based on the metadata, which can be used to find the relevant document.

In this project, Qdrant is used to store the document embeddings and the corresponding meta data. During the query processing, the system uses the database to find the relevant document chunks and generate an answer based on this information

2.9 Document Parsing

PDF files with government documents can have various structures with headings, paragraphs, tables, and pages with scanned images. Processing such documents requires careful extraction of text and preservation of the order of information. Additionally, the text in tables and scanned pages may need to be extracted separately.
The Unstructured library [17] provides several tools for document parsing, which can be used in this project. It can split the document into sections with different types of information, such as title, paragraph, and table.

The scanned pages need to be processed with Optical Character Recognition (OCR) to extract the text. The Tesseract engine can be used to perform this task.

This project uses the Unstructured library to process the PDF documents, which allows ex tracting the information with the proper structure. The OCR feature is used for scanned pages, where the text extraction would not be possible otherwise.

2.10 Comparison of Approaches

There are several approaches to providing answers to the questions about the documents. Key word search algorithms can find the relevant parts of the document, but they do not generate natural language answers. A language model can generate answers to the questions, but it may not have access to the required information from the given documents. The RAG approach explicitly retrieves the relevant information from the documents and uses this information to generate the answer. Table 2 compares these approaches.

Keyword search is useful for finding relevant parts of the document, but it is limited by the choice of keywords. It is possible that the user will ask the question with keywords which are not present in the document. The language model is capable of generating answers to the questions, but it is likely to generate unsupported answers when the required information is not found in the given documents. The RAG system can explicitly find and use the relevant information from the given documents. However, it relies on the quality of the retrieval and the ability of the LLM to use this information for generating accurate answers.

Table-2 Comparison of Approaches

Comparison Point	Keyword Search	LLM Only	RAG (This Project)
Understands semantic meaning	Limited	Can interpret meaning	Uses semantic retrieval and language generation
Uses the selected document collection	Yes	Not inherently	Yes
Generates natural-language answer	No, usually returns matching results	Yes	Yes

Comparison Point	Keyword Search	LLM Only	RAG (This Project)
Risk of unsupported answers	Low for direct matching, but results may be irrelevant	High when reliable context is unavailable	Reduced when retrieval and generation are properly controlled
Provides source information	Can identify matching documents	Not inherently	Can use retrieved metadata to identify sources
Requires model retraining when documents change	No	May require another knowledge source or model update	No, if only the document collection needs updating

2.11 Summary of the Survey

Surveying the relevant research, I identified three main aspects: the problem of hallucination, the RAGapproach, andtheimportanceofdocumentparsing. TheLLMsarepronetogenerating unsupported information, which is a major concern when the system is used to answer the

questions about the official schemes. RAG overcomes this limitation by using the documents as the source of truth, but this approach requires proper document processing. Based on the research, I focused on the processing of the government PDF documents, the generation of embeddings, and the use of instructions to prevent hallucination in the LLM.

The survey by Sharma [4] provided the general overview of different RAG approaches, which I used for selecting the methods for this project. Overall, the reviewed research findings directly impacted the decisions about the design of this project

Table 3: Summary of Research Papers on RAG and Dense Retrieval Evaluation

S. No.	Research Paper Name	Author(s)	Release/Publication Date	Problem Addressed
1	RAGAs: Automated Evaluation of Retrieval Augmented Generation	Es et al.	26 September 2023	Introduced metrics for assessing the relevance of retrieval and quality of answers.
2	RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models	Niu et al.	2024	Annotated dataset was provided for identifying hallucinations in answers generated using RAG.
3	Benchmarking Large Language Models in Retrieval-Augmented Generation	Chen et al.	2024	The robustness of RAG systems for noise was assessed along with evidence-based rejection and misinformation handling.
4	Dense Passage Retrieval for Open-Domain Question Answering	Karpukhin et al.	November 2020	Dense retrieval was introduced to locate semantically relevant passages.
5	BGE-M3: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation	Chen et al.	5 February 2024	Multilingual embeddings were developed for enabling semantic retrieval.

PROBLEM DEFINITION AND OBJECTIVES

Problem Statement

A significant amount of information is contained in PDFs, reports, guidelines, manuals and other such documents and publications from a variety of sources. However, sifting through this information and receiving accurate answers to specific questions can prove to be difficult and time consuming, now more than ever with an increase of unstructured information that users have to wade through to find the answers that they need. The following are some reasons why this can be so.

1. Information Hallucination: General purpose Large language models can hallucinate when queried about specific government schemes and scholarships.

2. No Source Reference: Many question-answering AI applications do not show the references of answers they generate. Thus making it hard to find out if the answer was provided by the model or with the proof from the documents.

3. Complex PDFs: Government documents may have scanned images of PDFs, tables, and multiple columns. Thus making it difficult to parse and extract the relevant information from them.

4. No support for unanswerable Questions: Many AI models try to answer any query provided to them but there might be cases where the information in the documents is not enough.

To solve the above-mentioned problems, there is a need to develop a Retrieval-Augmented Generation system with the help of which users can get relevant answers to their questions regarding government scholarships. The system will fetch required information from the given PDFs and show the answers with their references thus avoiding hallucination as much as possible. Also, the system can be trained to refuse an unknown query if the required information is not present in the PDFs.

Objectives

The purpose of this project is to design and evaluate a hallucination-resistant RAG system that provides answers based on government documents. The objectives of this experiment are as follows:

1. Increase the precision of the answers based on the relevant information extracted from the provided government documents.
2. Determine the retrieval performance of the system according to the set of test questions.
3. Ensure the answers provided by the model are supported by the retrieved documents with correct citations and page numbers
4. Determine the system's ability to identify and decline answering questions that cannot be answered using the provided documents.
5. Measure the hallucination rate of the system by determining the percentage of claims provided by the model that are not supported by the retrieved context.
6. Evaluate the system's reliability based on the retrieved hit rate, answer precision, answer citation validity, correct refusal rate, and hallucination rate.

PROPOSED METHODOLOGY AND WORK DESCRIPTION

4. Methodology

This section explains each step of the implementation. The code given here is a simplified form of the steps in my notebook.

4.1 Overview of the System

The system has two parts. The first part is the indexing part, which is done once for the doc uments. The second part is the question-answering part, which runs every time a user asks a question. Figure 1 shows the overall flow.

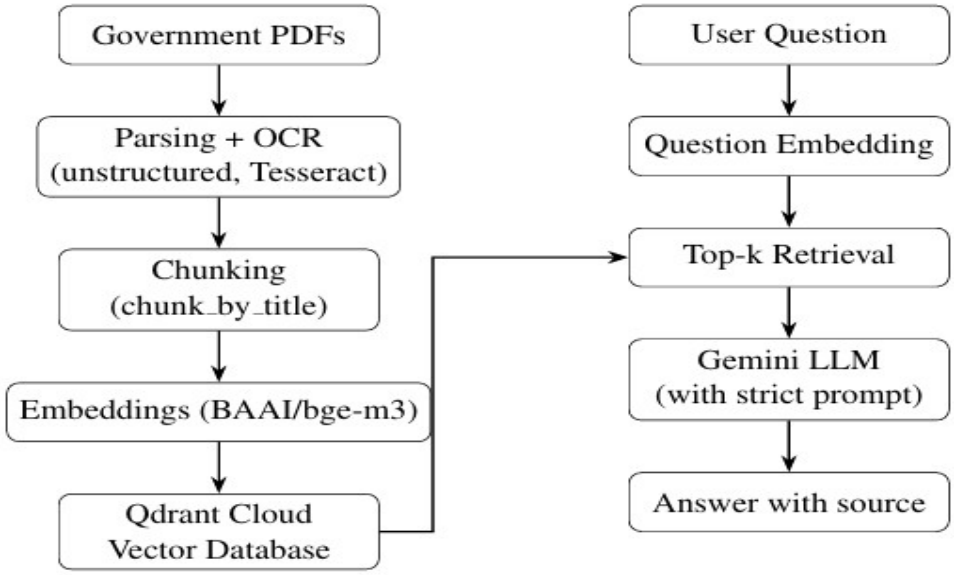


Fig-1 Overview of the System

4.2 Environment and Setup

The whole project was done in Python in a Jupyter notebook. The main libraries are unstructured for reading PDFs, sentence-transformers for the embedding model, qdrant -client for the database, and the Google Generative AI library for Gemini. For reading scanned pages, the Tesseract OCR program and the Poppler tools also need to be installed on the computer, because the **hi res** mode depends on them

The keys for Qdrant Cloud and for Gemini are secret, so they are not written directly in the notebook. They are kept in environment variables and read from there. This is a good habit, because if the notebook is shared, nobody else can use my keys

4.3 Documents Used

The system was built on Indian government scholarship documents in PDF form. The files are guidelines, notifications and circulars taken from official sources

4.4 Step 1: Reading the PDF Files

The scholarship PDFs are read using the partition library **unstructured** in **hi- res** pdf function of the res mode. In this mode, the library studies the page layout and finds titles, paragraphs, lists and tables. For scanned pages, Tesseract OCR is used to get the text from the images. Tables are kept as tables so that values like amounts and income limits do not get mixed up

4.5 Step 2: Chunking

The elements obtained from the PDF are grouped into chunks using the chunk method. It starts a new chunk whenever a new heading comes. Because of this, each chunk usually talks about one topic, like “Eligibility” or “Documents Required”. This is better than cutting the text after a fixed number of characters, because a rule is not broken in the middle of a sentence
Each chunk also keeps metadata such as the file name and the page number. This is later used to show the source of the answer.

4.6 Step 3: Creating Embeddings

Every chunk is converted into a vector using the **BAAI/bge-m3** model. This model understands meaning, supports many languages, and accepts long text. Two texts with similar meaning get vectors that are close to each other.

4.7 Step 4: Storing in Qdrant

All the vectors, along with their text and metadata, are stored in a Qdrant Cloud collection named government documents1. Qdrant uses cosinesimilarity to compare vectors. Since it is a cloud service, the data stays saved and does not need to be created again every time the notebook is run.

4.8 Step 5: Retrieval

When a user asks a question, it is converted into a vector using the same bge-m3 model. Qdrant then returns the top-k chunks whose vectors are closest to the question vector

4.9 Step 6: Answer Generation with llm model

The retrieved chunks are put into a prompt together with the question, and this prompt is sent to Google Gemini. The prompt gives clear rules to the model:

• Answer only from the given context.
• Donot use outside knowledge or guess.
• If the answer is not present in the context, say that the information is not available in the Documents
• Mention the source (file name and page) of the answer

A simple form of the prompt is shown below.

```
1. You are an assistant for Indian government
2. scholarship information.
3. Answer the question using ONLY the context below.
4. If the answer is not in the context, say:
5. "This information is not available in the
6. provided documents."
7. Do not guess. Mention the source of the answer.
8. Context:
9. {retrieved_chunks}
10. Question: {user_question}
```

4.10 Choice of Parameters

Some values in the pipeline can be changed, and each of them has an effect on the final answer. I tried to understand the reason behind each value and not just copy it.

Number of chunks (k). If k is small, there is a chance that the chunk with the answer is not in the list. If k is large, many unrelated chunks go to the model, the prompt becomes long, and the model may get confused. In my code, I started with k = 5, which is a common middle value, and it can be changed after looking at the test results.

Size of the chunks. Very small chunks lose the context, like the exception after a rule.Very large chunks contain many topics, and the search becomes less exact. Because I used Chunk by title, the size mostly follows the headings of the document.

Embedding size. The bge-m3 model gives a vector of 1024 numbers for each chunk, so the Qdrant collection is created with the size 1024. If the size given in the collection does not match the model, the upload fails.

Temperature. For question answering I kept the temperature of Gemini low, so that the answer does not change much each time and does not become creative.

Normalization. The vectors are normalized, so that the cosine similarity depends only on the direction of the vectors and not on their length.

4.11 How Hallucination is Reduced

The system tries to reduce hallucination in many ways at the same time:

1. Grounding: the model sees the real text from official documents.
2. Good parsing and chunking: tables and sections are kept complete, so the context is clean.
3. Strict prompt: the model is told not to guess and to refuse when the answer is missing.
4. Source citation: the user can check the answer in the original file.
5. Limited context: only the top-k related chunks are given, so the model is not confused by unrelated text.
6. Low temperature: a low temperature setting keeps the answers stable and fact-based.

PROPOSED ALGORITHMS

Algorithm 1:

Language model for hallucination-resistant retrieval-augmented generation for government information

Input: Government scheme PDF documents, query, embedding model, vector database, and language model

Output: A context-based answer with citations or default response for queries with no evidence in the corpus.

Phase 1: Document ingestion and knowledge base creation

Step 1: Document collection

The initial step involves collecting government documents about scholarships and other schemes and storing them in the input directory.

Step 2: PDF text extraction

The second step is to extract text, headings, tables, and other elements from the PDF documents. One has to choose an appropriate strategy for text extraction using the Unstructured library based on documents' structure. One also has to use OCR if necessary.

Step 3: Text cleaning

In this section clean the text by removing extra spaces, extra line breaks, and other irrelevant features while preserve relevant information, tables, headings, and scheme details.

Step 4: Text chunking

This section is to split the text into small chunks for easy processing. One has to make sure that chunks are of a reasonable length and have some overlap to retain context for the model. It is important to keep source information filename, page, and section for each chunk.

Step 5: Embeddings generation

The fifth step is to create document embeddings by encoding text chunks using the BAAI/bge-m3 model.

Step 6: Create vector database

The sixth step is to store embeddings in the Qdrant vector database. The database should have the right number of vector dimensions, and one should make sure to process each document only once to avoid extra chunks.

Phase 2: Query processing and answer generation

Step 7: Accept user query

The first step is to accept the user's query about government schemes, scholarships, eligibility, documents, benefits, and applications.

Step 8: Query encoding
The second step is to encode the query using the same encoder used to create document embeddings and the query encoding strategy.

Step 9: Retrieve relevant contexts
The third step is to retrieve relevant contexts from the vector database by querying with the encoded query.

Step 10: Check if the query has enough evidence
At this point, one has to review the chunks retrieved from the vector database and decide if there is sufficient evidence to answer the query. If there is no evidence, one has to output the default response:
"Information not found in the provided documents."

Step 11: Prompt creation
If there is enough evidence, one has to construct a prompt with the question and context chunks, specify how to use the context to answer the query, remind the model to avoid hallucinations, and ask it to identify different government schemes and their sources if applicable.

Step 12: Answer generation
The next step is to generate an answer to the query with the help of the Gemini language model. The answer should be based on evidence in the retrieved chunks.

Step 13: Add source citations
Finally, one has to add source citations to the answer and provide the answer to the user. If there is no evidence, one has to provide the default response.

Pseudocode:
Algorithm: Hallucination-Resistant RAG

Input: PDF documents D, user query Q

Output: Answer A with source references

1. Initialize embedding model E and vector database V.

2. For each PDF document d in D:

a. Extract text and elements using Unstructured.

b. Clean the extracted content.

c. Divide the content into overlapping chunks.

d. add source, page, and section metadata.

e. Generate embeddings using E.

f. Store chunks, embeddings, and metadata in Vector database.

3. Receive user query Q.

4. Generate query embedding q using E.

5. Retrieve top-ranked chunks C from V using q.

6. If C is empty or does not provide sufficient evidence:

Return "Information not found in the provided documents."

7. Construct a prompt P using Q, C, and grounding instructions.

8. Generate answer A using the Gemini language model.

9. Check whether A is supported by the retrieved context.

10. If A contains unsupported claims:

Reject or replace the unsupported answer with

"Information not found in the provided documents."

11. Otherwise, attach source references to A.

12. Return A and its source references.

PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

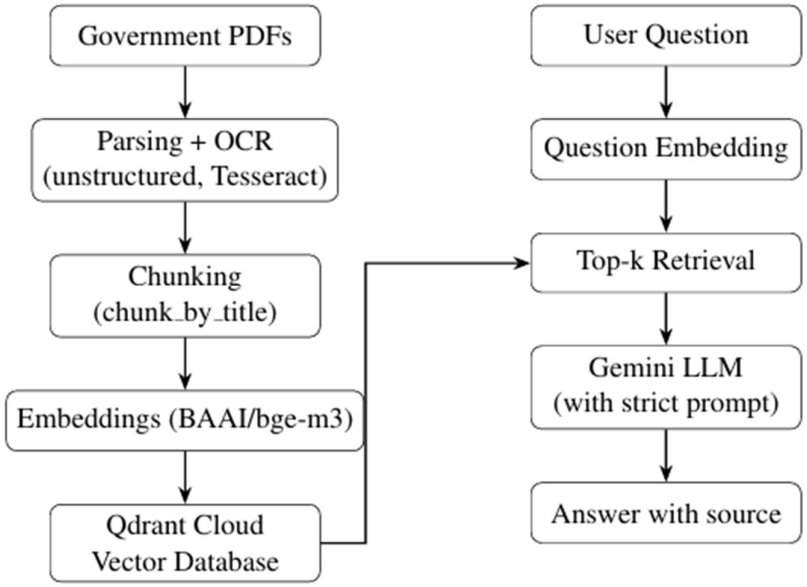


Fig 2: Architecture of the proposed system (left: indexing, right: question answering)

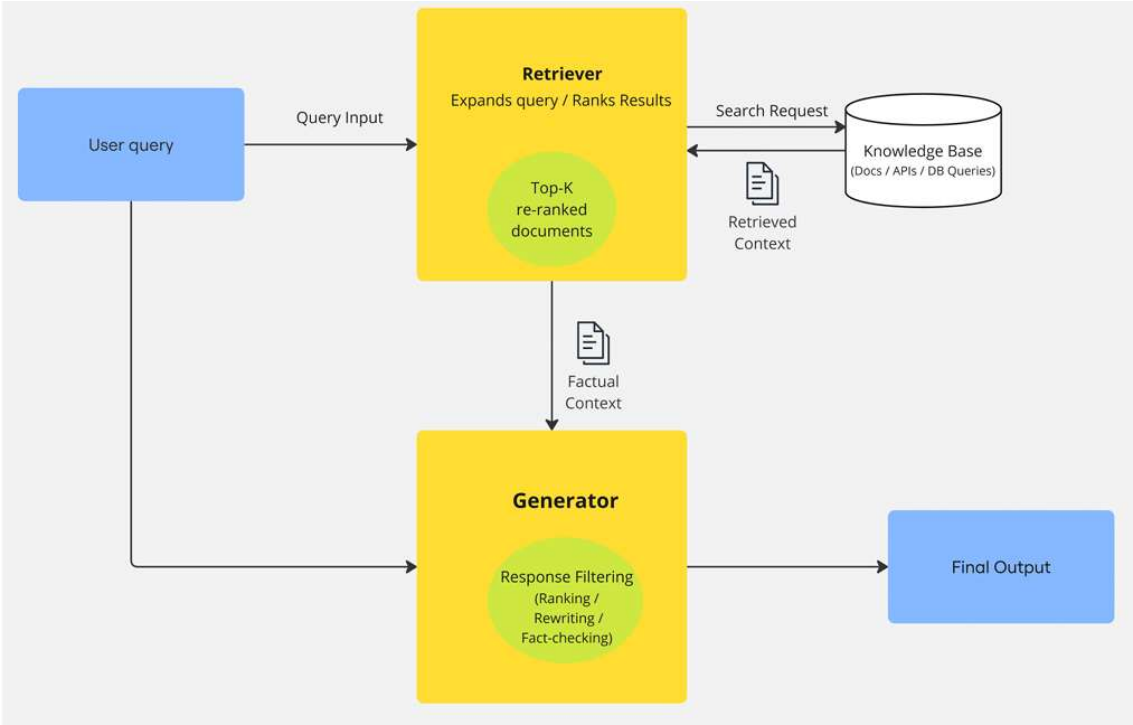


Fig. 3. Retrieval-Augmented Generation (RAG) workflow

IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

Table-4 TOOLS AND TECHNOLOGY USED

Component	Tool Used
Programming language	Python (Jupyter Notebook)
PDF parsing	unstructured (partition_pdf, hi-res)
OCR	Tesseract
Chunking	chunk_by_title
Embedding model	BAAI/bge-m3
Vector database	Qdrant Cloud
Language model	Google Gemini llm model

1. Programming Language – Python

The programming language Python was used to build the RAG pipeline and establish connections between its elements. The Jupyter notebook is chosen as the coding environment because it allows developing and testing the code conveniently in an organized way.

2. PDF Parsing – Unstructured

The library is used to extract the data from the provided PDF files. It can be achieved using the partition_pdf function that allows for faster processing in Hi-Res mode or other particular functions**.**

3. OCR – Tesseract

The tool is used to extract the text from the PDF file in case of scanned documents by performing OCR with high accuracy.

4. Chunking – Chunk by Title

Chunk by title is one of the options to split the whole text into several meaningful parts with the help of headers to retain the context during the process.

5. Embedding Model – BAAI/bge-m3

The embedding model is used to encode chunks and queries into vectors of numbers to find relevant text pieces during the search stage and overcome the challenges of lexical semantics.

6. Vector Database – Qdrant Cloud

The vector database is employed to store the embeddings with their corresponding text and metadata. After the query input, it is responsible for retrieving the relevant text chunks needed to respond to the user.

7. Language Model – Google Gemini llm model

The language model works on the context retrieved by the RAG system to provide the most relevant response. In this case, the model is fine-tuned to deny giving any information not

RESULT DISCUSSION AND ANALYSIS

6.1 Testing Method

To test the system, I asked questions related to the scholarship documents and checked each answer by comparing it with the original PDF. The questions were of three kinds:

1. Direct questions: the answer is clearly written in a document (for example, eligibility conditions or required documents).

2. Table-based questions: the answer is inside a table (for example, amounts or income limits).

1. **Out-of-document questions:** the answer is not present in the documents at all

6.2 Evaluation Measures

To judge the system in a fair way, I used the following simple measures

Retrieval hit rate: out of all the test questions, how many times the correct chunk was present in the top-k results

$$\text{Hit Rate} = \frac{\text{Questions with correct chunk in top-k}}{\text{Total questions}} \times 100$$

Answer accuracy: out of the questions whose answer is in the documents, how many answers were correct.

$$\text{Accuracy} = \frac{\text{Correct answers}}{\text{Answerable questions}} \times 100$$

Hallucination rate: how many answers contained information that is not supported by the documents

$$\text{Hallucination Rate} = \frac{\text{Unsupported answers}}{\text{Total answers}} \times 100$$

The survey [4] says that a RAG system should be judged on three main things: context rele vance (are the retrieved documents related to the question), answer faithfulness (is the answer based on the retrieved documents), and answer relevance (does the answer really reply to the question). My measures are close to these three. The hit rate shows context relevance, the hallucination rate shows answer faithfulness, and the accuracy shows answer relevance. The refusal rate is my extra measure, because for government information it is important that the

system says “not available” when it should. A similar idea of rejecting a question when the evidence is missing is also used in the RGB benchmark mentioned in the survey

6.3 Observed Results

Table 5: Results of the testing of the system

Measure	Result
Total questions tested	121
Retrieval hit rate	89.26%
Answer accuracy	82.64 %
Hallucination rate	5.79 %

6.4 Comparison with a Normal LLM

Table 6 shows how the answers of a normal LLM and the proposed RAG system differ for the three kinds of questions.

Table-6 Comparison with a Normal LLM

Question type	Normal LLM (without documents)	Proposed RAG system
Direct question	May give general or old information	Gives answer from the document with source
Table-based question	Often gives a guessed value	Gives value taken from the retrieved table chunk
Out-of-document question	Usually gives a made-up answer	Says the information is not available

6.5 Example Behaviour

The following examples show the way the system is expected to behave

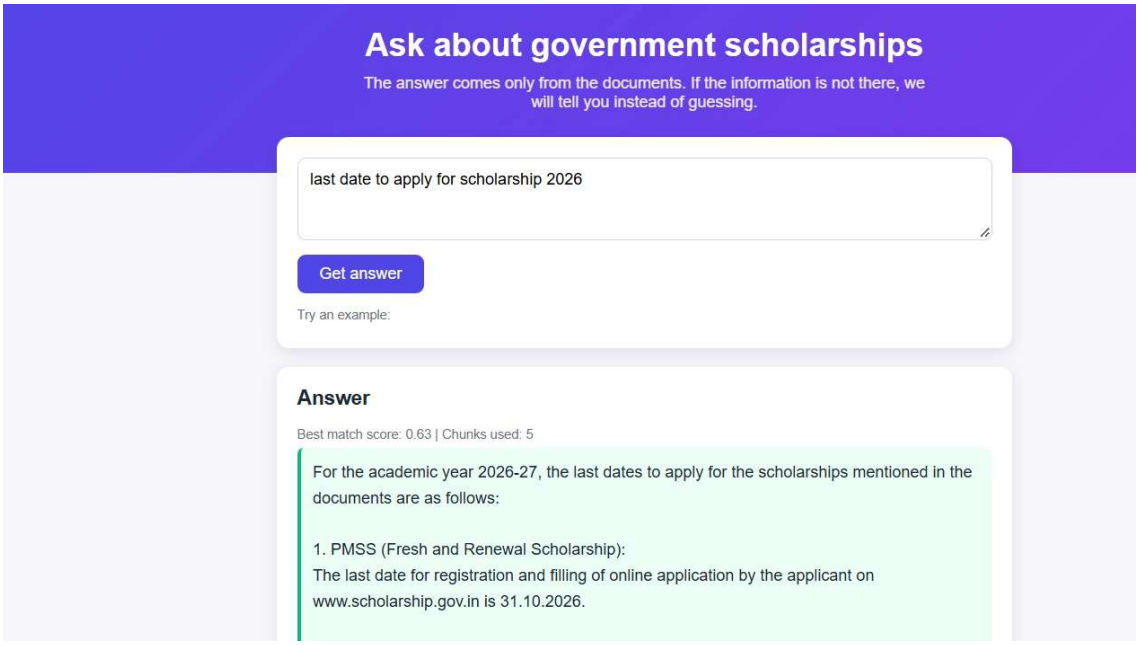


Fig-4 Output of the system when the answer is present in the documents

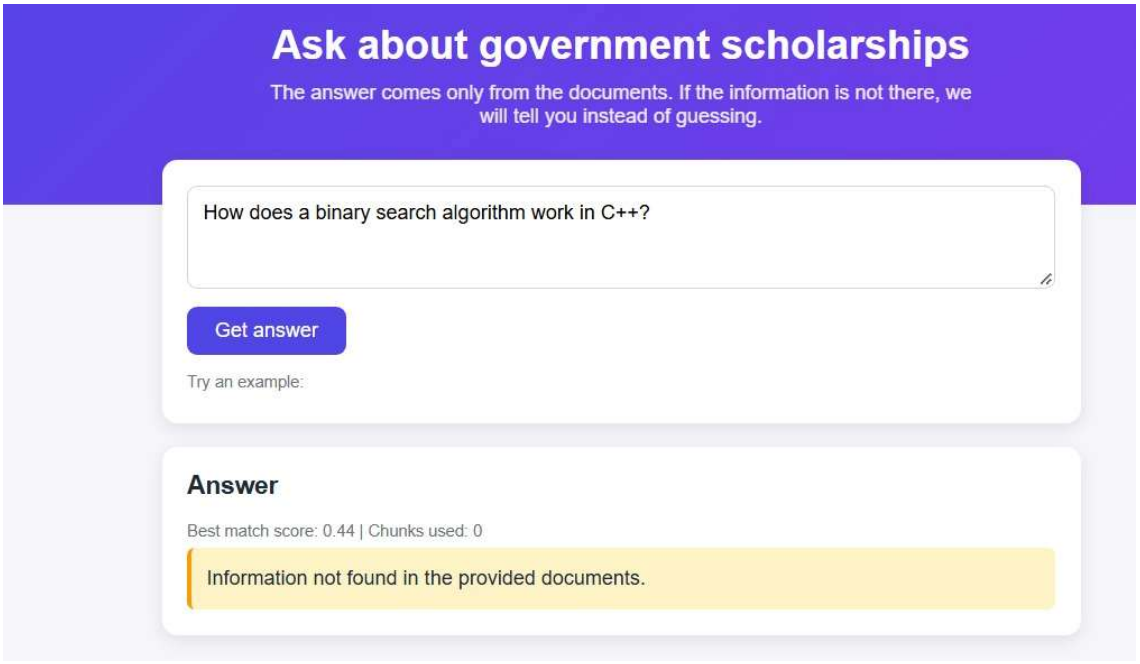


Fig-5 Output of the system when the answer is not present in the documents

6.8 Discussion

The results showed that the system works much better when the answer is present in the doc uments. Because the answers come with file name and page number, it is easy to verify them. The out-of-document questions were important in the test, because a good system should not answer when it has no information. With the strict prompt, the model mostly refused to answer instead of guessing.

I also noticed that the final answer depends a lot on retrieval. When the right chunk was not in the top-k results, the answer was incomplete or the model said that the information was not available. Scanned pages with bad quality also gave some OCR errors, which affected a few answers.

6.9 Error Analysis

The mistakes of the system can be divided into three groups.
1. Parsing errors: when a scanned page is blurry, OCR reads some words wrongly, and the wrong text goes into the database.
2. Retrieval errors: when the question is very short or unclear, the system may not find the right chunk

3. Generation errors: even with the right context, the model may sometimes join two facts in a wrong way, or give a shorter answer than needed

6.10 Problems Faced During the Project

While doing this project, I faced some practical problems.

• **Setting up the tools.** The hi res mode of unstructured needs Tesseract and Pop pler, and it took me some time to install them and make them work.

• **Slow processing**. Reading large PDFs with OCR takes a lot of time, so it is better to process the files once and store the result in the database.

• **Tables.** Some tables were not read perfectly, especially when the rows were very long.

• **Vector size mismatch**. At first, the collection size and the model output size must be the same, otherwise the upload gives an error.
• **Prompt writing**. It took a few tries to write a prompt in which the model follows the rule of refusing when the answer is missing.
• **API limits.** The free usage limits of the cloud services can slow down the work when many requests are sent together

6.11 Advantages of the System

• The answers come from official documents, so they are more trustworthy.
• The user can see the file name and the page number and check the answer.
• Newdocuments can be added without training any model.
• The system tells the user when it does not have the information. • It saves the time of reading long PDF files

6.12 Limitations

• OCR can make mistakes on blurred or low quality scans.
• The system depends on the quality of the retrieved chunks.
• It uses only the documents that were uploaded, so new notifications must be added to the database.
• The prototype is tested on a limited number of documents and questions.
• Hallucination is reduced, but a small chance of error can still remain with any LLM.

CONCLUSION AND FUTURE SCOPE

7 Conclusion

In this project, I made a RAG based question-answering system for Indian government schol arship documents. The system reads PDFs with OCR, makes topic-wise chunks, stores their embeddings in Qdrant, and uses Gemini to write answers only from the retrieved text. By using good document parsing, meaningful chunking, a strict prompt, and source citation, the system gives answers that are more reliable than a normal chatbot and refuses to answer when the information is not found.

During this work I learned that a RAG system is only as good as its weakest step. A good LLM cannot give a correct answer if the parsing is wrong or if the wrong chunk is retrieved. I also learned that it is better for a system to say "I do not know" than to give a wrong answer, especially for government information

This work also helped me to learn how LLMs, embeddings and vector databases work together in a real problem. I believe that such systems can help students find scholarship information quickly and with more trust

8 Future Scope

The present project is a prototype, and there are many ways in which it can be improved.

• **Hindi and regional languages**: allow questions and answers in Hindi and other Indian languages, so that more students can use the system.
• **Re-ranking: add a re-ranker** model that sorts the retrieved chunks again, so that the best chunk comes first.
• **Hybrid search**: combine keyword search (BM25) with vector search for better results, especially for exact terms like scheme names and form numbers.
• **Answer checking**: add a step that checks whether every sentence of the answer is sup ported by the context, and removes or flags the sentences that are not. Methods like SELF-RAG and CRAG, described in the survey [4], do a similar check and can be studied for this.
• **Testing against wrong documents**: test the system with noisy or misleading documents, because the survey shows that robustness to such documents is an important open prob lem.
• **Auto update**: automatically add new government notifications to the database, and re move old ones that are no longer valid.

• **Web and mobile app**: build a simple chatbot interface so that students can use it easily.
• **Voice support**: add speech input and output for users who find typing difficult.
• **Better evaluation**: prepare a bigger test set of questions and use automatic tools to measure the quality of the system.
• **User feedback**: add a thumbs up / thumbs down button, so that wrong answers can be noticed and the system can be improved.
• **Highlighting the source**: show the exact lines of the PDF from where the answer was taken.
• **Version control of documents**: keep the date of every document and prefer the latest one when two documents disagree.
• **Other departments**: extend the system to other government services like health, agri culture and pension schemes.

REFERENCES

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems (NeurIPS 2020).

[2] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. ACM Computing Surveys 55, 12, Article 248, pp. 1–38. DOI:https://doi.org/10.1145/3571730

[3] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval- Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997. https://arxiv.org/abs/2312.10997

[4] Chaitanya Sharma. 2025. Retrieval -Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers. arXiv:2506.00054. https://arxiv.org/abs/2506.00054

[5] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 6769-6781. Association for Computational Linguistics. DOI: https://doi.org/10.18653/v1/2020.emnlp-main.550

[6] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi- Granularity Text Embeddings via Self-Knowledge Distillation. arXiv:2402.03216. https://arxiv.org/abs/2402.03216

[10] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique via Self-Reflection. In The Twelfth International Conference on Learning Representations. https://openreview.net/forum?id=hSyW5go0v8

[11] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. arXiv:2401.15884. https://arxiv.org/abs/2401.15884

[12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics 12, pp. 157-173. DOI: https://doi.org/10.1162/tacl_a_00638

[13] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations. pp. 150-158. Association for Computational Linguistics. https://aclanthology.org/2024.eacl-demo.16/

[14] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. 2024. RAGTruth: A Hallucination Corpus for Building Trustworthy Retrieval-Augmented Language Models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 10862-10878. Association for Computational Linguistics. DOI: https://doi.org/10.18653/v1/2024.acl-long.585

[15] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. Benchmarking Large Language Models in Retrieval-Augmented Generation. In Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence. 38(16): 17754-17762. DOI: https://doi.org/10.1609/aaai.v38i16.29728

[16] Qdrant. N.D. Qdrant Documentation. https://qdrant.tech/documentation/

[17] Unstructured. N.D. Unstructured Documentation. https://docs.unstructured.io/

[18] Feiteng Fang, Yuelin Bai, Shiwen Ni, Min Yang, Xiaojun Chen, and Ruifeng Xu. 2024. Enhancing Noise Robustness of Retrieval-Augmented Language
Models with Adaptive Adversarial Training. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1:
Long Papers), Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10028–10039.
doi:10.18653/v1/2024.acl-long.540

[19] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval
augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997 2 (2023), 1.

[20] Michael Glass, Gaetano Rossiello, Md Faisal Mahbub Chowdhury, Ankita Naik, Pengshan Cai, and Alfio Gliozzo. 2022. Re2G: Retrieve, Rerank,
Generate. In Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language
Technologies, Marine Carpuat, Marie-Catherine de Marneffe, and Ivan Vladimir Meza Ruiz (Eds.). Association for Computational Linguistics, Seattle,
United States, 2701–2715. doi:10.18653/v1/2022.naacl-main.194